

# **ARP Poisoning Forensics**

**OJE DOMINION MAYOWA**

**FWSD2511507**

**Computer and Digital Forensics**

**MR. Aminu Idris**

**15 September 2026**

# LAB OVERVIEW

This lab examined ARP resolution behavior to distinguish normal traffic from potential spoofing activity. I captured a clean ARP exchange during a gateway ping, analyzed the request/reply field structure, then reviewed a separate capture for signs of ARP poisoning by checking for conflicting IP-to-MAC claims, unsolicited replies, and repeated MAC ownership across multiple IPs.

## OBJECTIVES

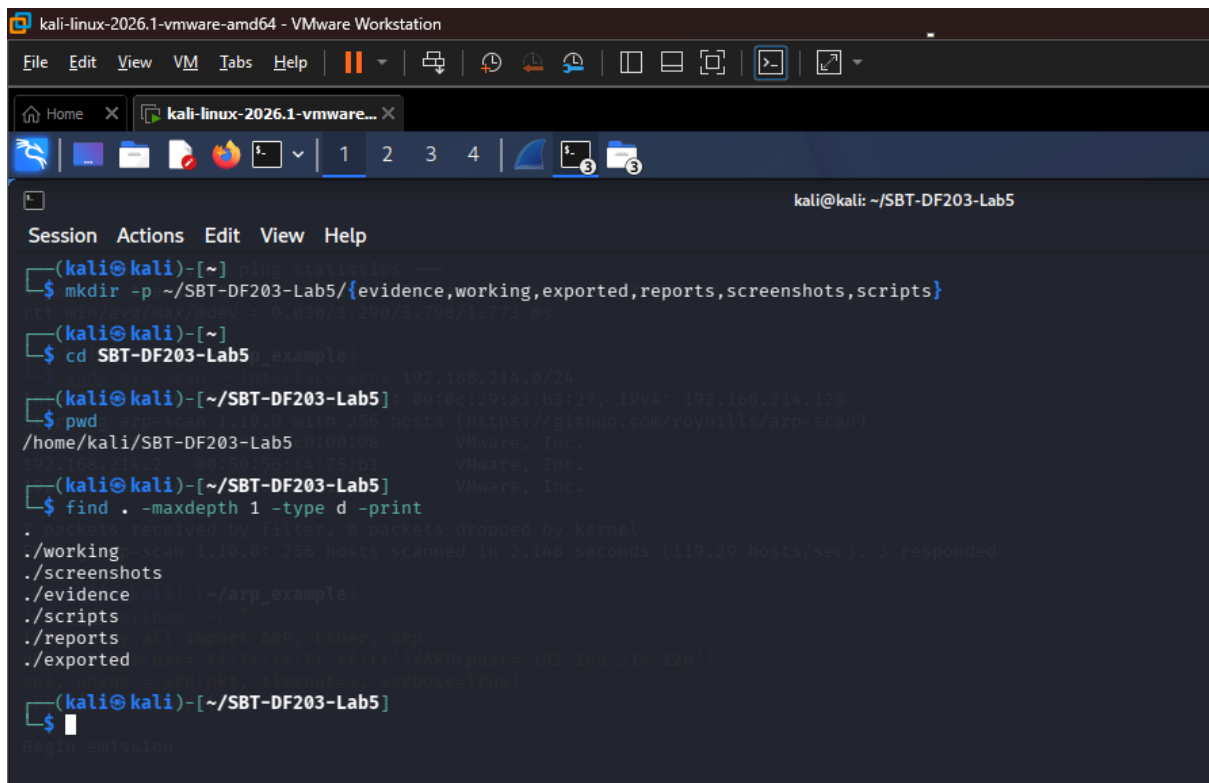
- Explain how ARP maps IPv4 addresses to MAC addresses on a LAN.
- Inspect ARP caches on Linux and Windows systems.
- Identify ARP requests, legitimate replies and unsolicited or conflicting replies.
- Detect one IP address being associated with multiple MAC addresses.
- Capture and document a bounded ARP-poisoning simulation in a host-only network.
- Restore network state and recommend preventive controls.

## TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- python3-scapy
- Md5sum and sha256sum
- ip neigh, arp, ipconfig /all
- Firefox
- Wireshark, tshark

# METHODOLOGY

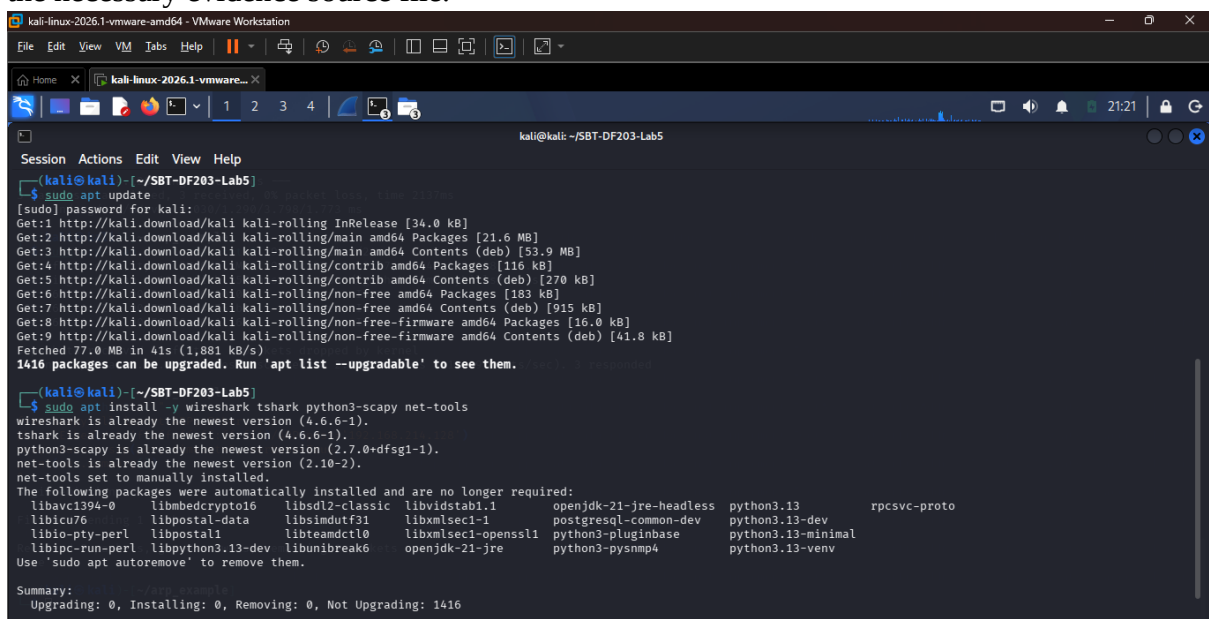
## Lab Folder Structure and Evidence Preparation



```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
(kali@kali)-[~]
$ cd SBT-DF203-Lab5
(kali@kali)-[~/SBT-DF203-Lab5]
$ pwd
/home/kali/SBT-DF203-Lab5
(kali@kali)-[~/SBT-DF203-Lab5]
$ find . -maxdepth 1 -type d -print
./working
./screenshots
./evidence
./scripts
./reports
./exported
```

Fig 1: Working Lab Created

Now we download the required tool for the successful running of the lab and we download the necessary evidence source file.



```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.9 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [41.8 kB]
Fetched 77.0 MB in 41s (1.881 kB/s)
1416 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~/SBT-DF203-Lab5]
$ sudo apt install -y wireshark tshark python3-scapy net-tools
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
net-tools is already the newest version (2.10-2).
net-tools set to manually installed.
The following packages were automatically installed and are no longer required:
 libavc1394-0 libbcbcrypt016 libbsd2-classic libbcbidstab1.1 openjdk-21-jre-headless python3.13 rpcsvc-proto
 libicu76 libpostal-data libsimdtf31 libxmlsec1-1 postgresql-common-dev python3.13-dev
 libio-pty-perl libpostal1 libteamdctl0 libxmlsec1-openssl1 python3-pluginbase python3.13-minimal
 libipc-run-perl libpython3.13-dev libunibreak6 openjdk-21-jre python3-pysnmp4 python3.13-venv
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1416
```

Fig 2: Updating the Lab environment and downloading necessary tools

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap'
--2026-09-17 21:26:41-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/Lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com) ... 185.199.109.133, 185.199.110.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap 100%[=====] 1.05K --KB/s in 0s

2026-09-17 21:26:43 (32.6 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Fig 3: Download the Source file into the Evidence Folder

Then I copied the Original source evidence to preserve the Integrity of the original source.

```

(kali@kali)-[~/SBT-DF203-Lab5] remaining 1 packets
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 4: Copied the Original source file to preserve the Original

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(kali@kali)-[~/SBT-DF203-Lab5]
$

```

To direct input to this VM, move the mouse pointer inside or press Ctrl+G.

Fig 5: Verifying the Hashes of both the Original and Copy

```

Ending arp-scan 1.10.0: 256 hosts scanned in 2.146 seconds (119.29 hosts/sec). 3 responded

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo                UNKNOWN          127.0.0.1/8 ::1/128
eth0              UP               192.168.214.128/24 fe80::cdf3:9b81:a87e:9111/64

```

Fig 6: ARP Table

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.214.2 dev eth0 proto dhcp src 192.168.214.128 metric 100
192.168.214.0/24 dev eth0 proto kernel scope link src 192.168.214.128 metric 100

```

Fig 7: route/gateway

```

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.214.2 dev eth0 lladdr 00:50:56:f4:75:b1 REACHABLE
192.168.214.254 dev eth0 lladdr 00:50:56:e0:11:6e STALE

(kali㉿kali)-[~/SBT-DF203-Lab5]
$

```

Fig 8: Interface addresses

## 7. Mini Evidence and Chain-of-Custody Worksheet

| Field                       | Student Entry                                                    |
|-----------------------------|------------------------------------------------------------------|
| Case/lab identifier         | SBT-DF203-Lab5-Oje Dominion                                      |
| Trainee name                | OJE DOMINION MAYOWA                                              |
| Date and time started       | 17/09/2026, 5:00PM                                               |
| Evidence file name(s)       | arp.pcap, working.pcap                                           |
| Source or generation method | Online Platform                                                  |
| Original SHA-256            | 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e |
| Working-copy SHA-256        | 342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e |
| Analysis workstation/VM     | Kali VM                                                          |
| Notes on any changes        |                                                                  |

## Part A - Observe Normal ARP Resolution

```

(kali㉿kali)-[~/SBT-DF203-Lab5]
$ IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
echo "Interface: $IFACE"
echo "Gateway: $GATEWAY_IP"
Interface: eth0
Gateway: 192.168.214.2

```

Fig 9: Setting Variable and confirming them

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
[sudo] password for kali:
$
```

Fig 10: Clear the ARP cache entry for the gateway

Then we start the capture and we leave it for a short duration of time for it to run

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
[1] 1031415
$
Capturing on 'eth0'

Received 0 packets, got 0 answers, remaining 1 packets
[1] + done tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng
```

Fig 11: Starting the Capture

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ sleep 2
ping -c 1 "$GATEWAY_IP"
PING 192.168.214.2 (192.168.214.2) 56(84) bytes of data:
64 bytes from 192.168.214.2: icmp_seq=1 ttl=128 time=1.51 ms
--- 192.168.214.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.507/1.507/1.507/0.000 ms
$
```

Fig 12: Generating the ARP resolution by pinging the gateway

I successfully generated the ARP by pinging the gateway if the client IP.

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ wait
ip neigh show | tee reports/arp_table_after_ping.txt
192.168.214.2 dev eth0 lladdr 00:50:56:f4:75:b1 REACHABLE
192.168.214.254 dev eth0 lladdr 00:50:56:e0:11:6e STALE
$
```

Fig 13: Showing the IPs available and their state

## Part B - Analyze ARP Request and Reply Fields

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
  -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/normal_arp_fields.tsv
1      2026-09-17T22:55:01.734880218+0100      00:0c:29:a1:b3:27      00:50:56:f4:75:b1      1      192.168.214.128 00:0c:29:a1:b3:27      192.168.214.2 00:00:
00:00:00:00
2      2026-09-17T22:55:01.736519559+0100      00:50:56:f4:75:b1      00:0c:29:a1:b3:27      2      192.168.214.2 00:50:56:f4:75:b1      192.168.214.128 00:0c:
29:a1:b3:27
3      2026-09-17T22:55:47.303407690+0100      00:0c:29:a1:b3:27      00:50:56:f4:75:b1      1      192.168.214.128 00:0c:29:a1:b3:27      192.168.214.2 00:00:
00:00:00:00
4      2026-09-17T22:55:47.304909161+0100      00:50:56:f4:75:b1      00:0c:29:a1:b3:27      2      192.168.214.2 00:50:56:f4:75:b1      192.168.214.128 00:0c:
29:a1:b3:27
(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Fig 14: Analyzing ARP Request and their respective reply Fields

| Field                   | Normal Request                    | Normal Reply                          |
|-------------------------|-----------------------------------|---------------------------------------|
| Ethernet destination    | 00:50:56:f4:75:b1                 | 00:0c:29:a1:b3:27                     |
| ARP opcode              | 1                                 | 2                                     |
| Sender protocol address | 192.168.214.128                   | 192.168.214.2                         |
| Sender hardware address | 00:0c:29:a1:b3:27                 | 00:50:56:f4:75:b1                     |
| Target protocol address | 192.168.214.2                     | 192.168.214.128                       |
| Forensic interpretation | 192.168.214.128 has 192.168.214.2 | 192.168.214.2 is at 00:50:56:f4:75:b1 |

## Part C - Analyze the Supplied Poisoning Capture

```

(kali@kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap
# Inventory all ARP claims
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65
(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Fig 15: ARP packet capture and analysis using TShark

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ # Summarize IP-to-MAC claims
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

Fig 16: Summarizes the IP to MAC address

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff' -T fields \
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
| tee reports/unicast_arp_replies.tsv
4 2023-03-12T04:22:44.904123642+0100 136.160.215.194 00:50:56:86:02:65 00:50:56:86:cb:fc
6 2023-03-12T04:22:49.943043799+0100 136.160.215.15 00:50:56:86:cb:fc 00:50:56:86:02:65

(kali㉿kali)-[~/SBT-DF203-Lab5]
$
```

Fig 17: Looking for unsolicited replied from IPs

## Part E - Detection Logic and Timeline

| Time | Claimed IP | Claimed MAC | Target | Request Seen First? | Assessment |
|------|------------|-------------|--------|---------------------|------------|
|      |            |             |        |                     |            |

|                           |                 |                   |                 |                                              |                                                                           |
|---------------------------|-----------------|-------------------|-----------------|----------------------------------------------|---------------------------------------------------------------------------|
| <b>04:22:03.059513401</b> | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.1   | N/A<br>(gratuitous<br>self-<br>announcement) | Normal —<br>host<br>announcing its<br>own presence                        |
| <b>04:22:38.192015486</b> | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.199 | N/A (this IS a<br>request)                   | Normal — .1<br>asking who<br>has .199; no<br>reply seen in<br>this window |
| <b>04:22:44.903581555</b> | 136.160.215.15  | 00:50:56:86:cb:fc | 136.160.215.194 | N/A (this IS a<br>request)                   | Normal — .15<br>asking who<br>has .194                                    |
| <b>04:22:44.904123642</b> | 136.160.215.194 | 00:50:56:86:02:65 | 136.160.215.15  | Yes (frame 3,<br>0.0005s<br>earlier)         | Normal —<br>timely reply<br>matching the<br>request                       |
| <b>04:22:49.943023192</b> | 136.160.215.194 | 00:50:56:86:02:65 | 136.160.215.15  | N/A (this IS a<br>request)                   | Normal —<br>.194 asking<br>who has .15                                    |
| <b>04:22:49.943043799</b> | 136.160.215.15  | 00:50:56:86:cb:fc | 136.160.215.194 | Yes (frame 5,<br>immediately<br>prior)       | Normal —<br>timely reply<br>matching the<br>request                       |
| <b>04:23:03.059262685</b> | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.1   | N/A<br>(gratuitous<br>self-<br>announcement) | Normal —<br>repeat self-<br>announcement,<br>consistent<br>MAC            |
| <b>04:23:06.191965551</b> | 136.160.215.1   | 00:1b:17:00:0a:30 | 136.160.215.183 | N/A (this IS a<br>request)                   | Normal — .1<br>asking who<br>has .183                                     |

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 \
| tee reports/all_arp_replies_timeline.txt
4      2023-03-12T04:22:44.904123642+0100      136.160.215.194  00:50:56:86:02:65      136.160.215.15
6      2023-03-12T04:22:49.943043799+0100      136.160.215.15  00:50:56:86:cb:fc      136.160.215.194

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac | sort | uniq -c | sort -k2
1 136.160.215.15 00:50:56:86:cb:fc
1 136.160.215.194 00:50:56:86:02:65

(kali@kali)-[~/SBT-DF203-Lab5]
$ tshark -r "$PCAP" -Y 'arp' -T fields -e frame.number -e frame.time -e arp.opcode -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 \
| tee reports/full_arp_sequence.txt
1      2023-03-12T04:22:03.059513401+0100      1      136.160.215.1  00:1b:17:00:0a:30      136.160.215.1
2      2023-03-12T04:22:38.192015486+0100      1      136.160.215.1  00:1b:17:00:0a:30      136.160.215.199
3      2023-03-12T04:22:44.903581555+0100      1      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194
4      2023-03-12T04:22:44.904123642+0100      2      136.160.215.194 00:50:56:86:02:65      136.160.215.15
5      2023-03-12T04:22:49.943023192+0100      1      136.160.215.194 00:50:56:86:02:65      136.160.215.15
6      2023-03-12T04:22:49.943043799+0100      2      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194
7      2023-03-12T04:23:03.059262685+0100      1      136.160.215.1  00:1b:17:00:0a:30      136.160.215.1
8      2023-03-12T04:23:06.191965551+0100      1      136.160.215.1  00:1b:17:00:0a:30      136.160.215.183

(kali@kali)-[~/SBT-DF203-Lab5]
```

**Fig: Detection Logic Evidence**

**In conclusion,** the analyzed capture showed no evidence of ARP spoofing: every IP address was consistently claimed by a single, stable MAC address, all replies matched a prior request, and no gateway MAC changes were observed. This confirmed normal, legitimate ARP behavior across the network.